

Next-Generation Agentic UI/UX Integration and Machine-Readable Infrastructure for the Cianfhoghlaime Platform

Architectural Context and Baseline of the Cianfhoghlaime Monorepo

The cianfhoghlaime repository represents a highly sophisticated, multi-nation, and multi-language data platform specifically engineered for Celtic education.¹ The foundational architecture of the backend and data engineering ecosystem is exceptionally robust, built around a modern, five-stage ingestion-to-exposure pipeline orchestrated by Dagster.² This orchestration framework is governed by a strict five-layer component architecture where each component is registered in the pyproject.toml under [tool.dg].registry_modules, allowing the dg list components command to dynamically discover them.³ This heavily automated backend seamlessly moves data from varied sources—such as PDFs, DOCX files, EPUBs, and Google Takeout—through a highly structured pipeline.²

The ingestion phase utilizes Data Load Tool (DLT) sources to move raw data into a DuckLake architecture, which serves as an ACID-compliant lakehouse utilizing Parquet on Garage S3 with a Postgres catalog.² The extraction phase leverages BAML for structured schema extraction, which subsequently feeds into the embedding phase utilizing CocolIndex v1 and LanceDB for Hierarchical Navigable Small World (HNSW) vector indexing.¹ The cognition phase builds a dense knowledge graph via Cognee, and finally, the exposure phase makes the data accessible to end-users and agents alike.² The platform is further empowered by an advanced optical character recognition (OCR) registry containing nine vision models, four classical models, and three image-generation models, directly supporting the OCR-aware flow and the leabharlann corpus pipeline.²

This data infrastructure is ultimately exposed to a sophisticated fleet of twelve autonomous agents.² This fleet includes specialized agents for curriculum modeling, translation, corpus interaction, research, geospatial analysis, voice synthesis, statistics, education-research, and a root orchestrator.² The agents operate utilizing extensive memory systems composed of Cognee, Graphiti, LanceDB, FalkorDB, and Memgraph, ensuring long-term contextual retention.²

While the backend infrastructure demonstrates state-of-the-art data processing and agentic capabilities, the frontend ecosystem is currently undergoing a massive architectural shift to match this level of sophistication. Historically, the platform managed multiple disparate web

interfaces with overlapping dependencies, resulting in a thirty to forty percent code duplication overlap.⁵ To eliminate this technical debt and streamline the authentication mechanisms via a unified Pocket ID Single Sign-On (SSO) system, a comprehensive web application consolidation plan has been initiated.⁵

The target architecture for the frontend is a unified monorepo application located at cianfhoghlaime/web/cio-web/.⁵ This unified target will utilize TanStack Start and React—mirroring the stack of the legacy oideachais-web application—and will feature an embedded Hono API gateway functioning as a single deployable unit.⁵ To mitigate the risks associated with large-scale deployments, the migration strategy explicitly avoids "big-bang" cutovers in favor of a sequential, domain-by-domain migration.⁵

Domain Name	Current Repository Path	Target Monorepo Path	Risk Level	Estimated Effort	Sequence
hono-api	web/hono-api/	web/cio-web/api/	Low	Small (1 week)	1st
croilar-web	web/apps/croilar-web/	web/cio-web/croilar/	Low	Medium (1-2 weeks)	2nd
croilar-portal	web/apps/croilar-portal/	web/cio-web/croilar/admin/	Low	Small (1 week)	3rd
tuatha-ui	web/apps/tuatha-ui/	web/cio-web/tuatha/	Medium	Medium (2 weeks)	4th
oideachais-web	web/apps/oideachais-web/	web/cio-web/	High	Large (3-4 weeks)	5th

The primary challenge currently facing the repository is bridging the structural gap between the heavily automated, agent-driven backend and this newly consolidating frontend. Integrating informed agent skills into the user interface and user experience (UI/UX) design requires moving beyond traditional human-centric development workflows. It necessitates the adoption of machine-readable infrastructure, design-tokens-as-code pipelines, and Model Context Protocol (MCP) servers that allow state-of-the-art AI models to autonomously generate, test, and self-heal user interfaces without violating established design systems or generating unusable code.

The State of the Art in Agentic UI/UX Design

By 2026, the landscape of UI and UX design has undergone a profound paradigm shift, moving away from manual, highly iterative design cycles toward instantaneous, agent-assisted generative processes.⁶ Artificial intelligence is no longer merely a copilot feature utilized for drafting boilerplate code; it has evolved into a fully autonomous operational layer capable of executing end-to-end frontend workflows.⁸ Agents are now explicitly designed to interpret high-level business goals, break them down into granular component architectures, utilize available Application Programming Interfaces (APIs), and independently evaluate their outputs for quality control before ever presenting the code for human review.⁸

In the current state of the art, several generative AI platforms have redefined the design-to-code pipeline. Tools such as Moonchild AI function as intelligent product design partners, understanding deep contextual constraints and adapting to specific project moods to generate high-quality interfaces rapidly.⁷ For rapid prototyping and ideation, platforms like Galileo AI facilitate highly accurate text-to-UI generation, while Uizard specializes in converting rudimentary whiteboard sketches directly into functional screen layouts.⁷ Within the developer ecosystem, tools like v0.dev by Vercel have become the industry standard for instantly generating production-ready React components.⁷ Leveraging established utility libraries and component systems like Tailwind CSS and shadcn/ui, these generative platforms deliver clean, modern, and highly usable code that integrates seamlessly into existing React architectures.⁹ However, the second-order implication of this rapid generative capability is a critical crisis of governance and consistency. If AI agents generate UI components in isolation, lacking awareness of a centralized overarching design system, the result is a fragmented, inconsistent user experience commonly referred to in the industry as "AI slop".¹⁰ When an agent is not anchored to a repository's specific design logic, it will inevitably hallucinate new patterns, introduce severe technical debt, and violate accessibility standards.¹⁰ To circumvent this degradation, the industry has aggressively shifted toward rendering design systems fully machine-readable.¹² Instead of relying on human intuition to interpret brand guidelines from a static wiki, design teams now encode visual consistency directly into the development pipeline. The demand for Senior Front-End developers has surged, specifically for specialists who possess fluency in AI UI generation, accessible component primitives, and design-tokens-as-code.¹⁴ These specialists build the infrastructure that ensures agents remain trustworthy, governable, and composable across the enterprise.¹⁵ The future of UI/UX lies not in utilizing isolated generation tools via a browser interface, but in integrating these cognitive models directly into the repository's continuous integration and design systems via standardized programmatic protocols.

Establishing Infrastructure as Code for Design: Style Dictionary v4 and Tailwind CSS

To align the cianfhoghlaím frontend with the highly structured nature of its data engineering

backend, the foundational aesthetic layer of the UI/UX requires a robust Infrastructure as Code (IaC) approach. This is achieved through a strict "Design Tokens as Code" pipeline.¹⁶ Design tokens are machine-readable representations of a brand's design language, encompassing discrete variables such as color palettes, font families, sizing scales, elevation shadows, and animation durations.¹⁸ By defining these baseline variables in a platform-agnostic format like JSON or YAML, the repository ensures absolute visual consistency across diverse outputs, mathematically guaranteeing that the TanStack web application and the Tuatha MMO share the exact same aesthetic foundation.¹⁸

The enterprise standard for managing, transforming, and distributing these tokens is Style Dictionary, which has recently undergone massive architectural improvements culminating in its Version 4 release.¹⁹ Style Dictionary v4 natively supports modern ECMAScript Modules (ESM), making it browser-compatible out of the box and introducing support for asynchronous execution across its various APIs.²⁰ This allows the token engine to integrate perfectly with modern build tools like Vite, which powers the TanStack Start environment.²¹ Furthermore, Style Dictionary v4 aligns seamlessly with the Design Tokens Community Group (DTCG) specification.¹⁹ This specification standardizes the JSON format for design tokens globally, utilizing structures like \$value and \$type rather than proprietary nesting conventions.²³ For the cianfhoghlaím monorepo, which relies heavily on utility-first styling paradigms, Style Dictionary must be configured to transform raw JSON design tokens directly into a Tailwind CSS v4 configuration.¹⁸ This integration creates a unidirectional, immutable flow of design truth. An AI agent, or a human designer operating in a tool like Figma, defines or updates a design token. This token is exported as a strict JSON file and committed to the repository via the GitOps pipeline. The Style Dictionary build process then automatically ingests this JSON, applies pre-configured formatters, and outputs CSS variables, SCSS files, or JavaScript objects tailored specifically for the Tailwind configuration.¹⁸

Style Dictionary Tailwind Attribute	Expected Type	Functional Description	Default Value
type	'all' or string	Sets the name of each theme group (e.g., colors, fontSize) for Tailwind	-
formatType	'js' or 'cjs'	Sets the specific module format of the output Tailwind CSS configuration file	js

isVariables	boolean	Determines if native CSS custom variables should be generated and utilized	false
extend	boolean	Determines whether to append transformed styles to the 'extend' key within the theme	true
source	Array of strings	Specifies the file paths to the source design token JSON files	['tokens/**/*.json']

The table above illustrates the typical configurations required to bridge the Style Dictionary compilation process with the Tailwind CSS engine.²⁴ Integrating this specific IaC pipeline dictates that an AI agent tasked with creating a new functional feature in the croilar-portal domain does not hallucinate or hardcode hex values. Instead, it queries the machine-readable token registry, applies the correct semantic class (such as text-brand-primary or bg-surface-elevated), and relies on the Style Dictionary engine to resolve the underlying value during the build phase. This architecture not only eliminates visual regressions across the consolidated web applications but also provides autonomous agents with a heavily restricted, safe generative playground where design guardrails are mathematically enforced at compile time.²⁵

Machine-Readable Component Architectures: The Storybook Model Context Protocol

While Style Dictionary and design tokens govern the foundational aesthetic variables, the actual composition, logic, and testing of user interface components require a higher-level governance structure. This paradigm shift requires the implementation of an Agentic Design System.¹² Traditional design systems—typically documented in wikis, static HTML pages, or human-readable markdown—are fundamentally incompatible with autonomous agents. When an AI agent lacks explicit, structured, and queryable knowledge of existing repository components, it invariably hallucinates new patterns, duplicates existing functionality, introduces technical debt, and violates complex accessibility standards.¹⁰ An Agentic Design System solves this critical failure point by encoding design intent, component relationships, and usage

constraints into a strictly machine-readable context.¹² This enables AI agents to observe, detect, suggest, fix, and learn within an automated self-healing loop under human oversight.¹² The technological cornerstone enabling this machine-readable infrastructure is the Model Context Protocol (MCP).²⁶ MCP is an open standard, originally developed by Anthropic and now widely adopted, designed to facilitate secure and structured communication between AI agents and external operational tools.²⁶ For frontend architectures utilizing React and TanStack Start, the Storybook MCP server represents the absolute state-of-the-art integration for agentic UI design.¹³ Storybook, which serves as a widely adopted frontend workshop for building UI components in isolation, has evolved to expose its entire internal content and functional capabilities as a set of callable tools for AI agents.²⁶

Rather than relying on naive prompt engineering—where an entire codebase is dumped into a model's context window, which is computationally inefficient and prone to severe hallucination—Storybook intelligently generates a Component Manifest.²⁸ This manifest acts as a hyper-optimized payload that packages component APIs, validated prop types, usage examples extracted directly from living stories, JSDoc descriptions, and comprehensive test suites into a format specifically designed for Large Language Model consumption.³⁰ When integrated into a repository, the MCP server provides the AI agent with three critical, distinct toolsets: documentation, development, and testing.²⁷ The documentation toolset allows the agent to dynamically discover which components exist within the design system and fetch explicit, up-to-date usage guidelines.²⁸ The development toolset, utilizing specific endpoints such as `get-storybook-story-instructions`, provides the agent with strict architectural instructions detailing correct import paths, play function patterns, mocking strategies, and a hard programmatic rule demanding the execution of `run-story-tests` after every single component modification.²⁸ Furthermore, it exposes the `preview-stories` endpoint, which returns live, rendered story URLs.²⁸ For agent interfaces that support MCP Apps, these rendered components embed directly into the chat interface, allowing the human developer to visually verify the agent's work—including complex interactions like hover states—without ever leaving their IDE or terminal environment.¹⁰

Perhaps the most revolutionary aspect of the Storybook MCP integration is the testing toolset, which enables a completely autonomous self-healing loop.¹¹ Through the `run-story-tests` tool, the agent can execute interaction and accessibility tests against the React stories it just generated, utilizing a Vitest-powered test runner.²⁸ The causal loop operates as follows: if a test fails—for instance, if an agent-generated submit button within the `oideachais-web` application lacks adequate color contrast according to WCAG standards—the agent receives a structured accessibility violation report. It then autonomously queries the documentation toolset for the correct, accessible brand color token, utilizes its editing tools to update the React component, and re-runs the test suite.³² This iterative cycle repeats until all tests pass, ensuring that any code the agent presents for human review is already mathematically guaranteed to be functional, accessible, and compliant with the repository's strict design standards.¹¹

The implementation of Storybook MCP directly closes the critical gap between raw generative AI capability and enterprise-grade repository governance.²⁸ Developers integrating this toolset

have two primary architectural options. The first is the official @storybook/addon-mcp, published directly by the Storybook core team, which operates alongside the standard Storybook development server on a local port.¹⁰ The second is the community-built alternative, storybook-mcp (developed by mcpland), which takes a different architectural approach utilizing headless browser automation via Playwright to extract documentation in real-time from any deployed Storybook instance.²⁷ For the cianfhoghlaím monorepo, initializing the official @storybook/addon-mcp within the local TanStack workspace provides the most direct and reliable access to the self-healing test loop during the development phase.³¹

Orchestrating the UI Testing Pipeline via Dagger

To ensure the reliability of both human-authored and agent-generated code across the cianfhoghlaím repository, the infrastructure must transition away from legacy, YAML-based Continuous Integration and Continuous Deployment (CI/CD) pipelines toward a fully programmable execution environment. This transition is facilitated by Dagger, a modern CI/CD engine created by the founders of Docker, which allows developers to orchestrate highly complex test pipelines using standard programming languages.³⁵ Dagger fundamentally solves the "push and pray" dilemma inherent in traditional CI tools by allowing pipelines to run natively in sandboxed containers on local machines, AI sandboxes, or remote cloud servers with absolute guaranteed repeatability.³⁵

The cianfhoghlaím platform already utilizes Dagger for its polyglot CI/CD pipelines, bridging Python and TypeScript via an intricate 8-step GitOps process.⁴ To support the new Agentic UI/UX workflow and the Storybook MCP integrations, this existing Dagger infrastructure must be significantly expanded to handle frontend testing tools natively. The Daggiverse ecosystem provides dedicated SDK modules for both TypeScript and Python, enabling direct, programmatic integration with essential frontend testing frameworks.³⁸ Using these modules, the repository can instantiate isolated container environments to run ESLint for static analysis, Prettier for formatting, and Vitest for executing the interaction tests generated by the Storybook MCP.³⁹

Daggiverse Module	Target Ecosystem	Primary Orchestration Purpose
Pytest	Python	Orchestrates unit and integration tests for the backend Python agents and Dagster pipelines. ⁴⁰
Vitest	JavaScript / TypeScript	Executes the interaction and accessibility tests generated by the AI agent

		via Storybook MCP. ³⁹
ESLint	JavaScript / TypeScript	Enforces strict static code analysis and linting rules on all newly generated React components. ³⁹
Prettier	Multi-language	Ensures all agent-generated code conforms strictly to the repository's formatting guidelines. ³⁹

When an AI agent proposes a pull request to the newly consolidated web/cio-web/ application, the Dagger pipeline initiates a containerized build that executes the exact same run-story-tests protocol utilized during local development.²⁸ Because Dagger explicitly defines host dependencies and caches intermediate artifacts just-in-time, the remote test environment is perfectly identical to the local environment in which the AI agent originally generated and validated the code.³⁵

Furthermore, integrating visual regression testing tools, such as Chromatic, into the Dagger pipeline provides an additional layer of security.⁴² While Vitest handles programmatic interactions, Chromatic automatically captures high-fidelity screenshots of the agent-generated UI components across various states, themes, and viewports, comparing them against established visual baselines.²⁵ This multi-layered validation strategy—where Storybook MCP enforces component correctness locally, Style Dictionary enforces token integrity mathematically, and Dagger enforces pipeline integrity remotely—ensures that the consolidated web application remains highly stable despite the rapid velocity of AI-driven iteration. The entire Dagger definition can be rigorously validated using the local CLI via `dg check toml`, ensuring that the `pyproject.toml` and state-backed components remain free of syntax errors before the pipeline executes.⁴³

The Cognitive Engine: MiniMax M3 Architecture and Capabilities

Executing these sophisticated IaC, testing, and UI generation workflows requires a foundational AI model possessing extraordinary reasoning capabilities, flawless coding syntax generation, and unprecedented context-retention depth. For the cianfhoghlaime repository, the MiniMax M3 model serves as the ideal cognitive engine for these tasks. Released as an open-weight model, MiniMax M3 systematically outperforms leading competitors such as Opus 4.7 and GPT 5.5 in critical engineering benchmarks while remaining fifty times more cost-effective to operate.⁴⁵ The architectural specifications of the MiniMax M3 model represent a significant leap in generative technology. The model boasts 428 billion total parameters, utilizing a native

multimodal Mixture-of-Experts (MoE) network architecture.⁴⁶ Because it was trained natively on text, images, and video from the very first step—rather than having multimodality appended post-training—it achieves a much deeper semantic fusion across inputs.⁴⁶ During inference, approximately 22 billion parameters (A22B) are activated per token, ensuring rapid processing speeds while maintaining the depth of reasoning required for complex architectural tasks.⁴⁶ However, the most transformative feature of the MiniMax M3 model regarding the cianfhoghlaím integration is its massive one-million token context window, supported by a guaranteed operational minimum of 512,000 tokens.⁴⁹ This extreme context length is made computationally viable by the proprietary MiniMax Sparse Attention (MSA) architecture.⁴⁹ Unlike standard grouped-query attention (GQA) architectures that suffer from quadratic computational scaling, the MSA architecture utilizes a highly optimized pre-filtering stage.⁴⁶ This stage rapidly identifies relevant context blocks and attends strictly to those, reading each key-value cache block exactly once with contiguous memory access.⁴⁶ This architectural breakthrough delivers up to a nine-fold speedup in prefilling and a fifteen-fold speedup in decoding compared to previous generations, drastically reducing the per-token compute requirement to one-twentieth of its predecessor, all without compressing key-values or sacrificing precision via its BF16 and MXFP8 formats.⁴⁶

Model Specification	Technical Detail
Model Designation	MiniMax M3
Network Architecture	Mixture-of-Experts (MoE), Native Multimodal
Total Parameters	428 Billion
Active Parameters	~22 Billion per token (A22B)
Context Length	Up to 1,000,000 tokens
Attention Mechanism	MiniMax Sparse Attention (MSA)
Precision Formats	BF16, MXFP8

For the cianfhoghlaím project, a one-million token context window is not merely a convenience; it is a fundamental infrastructural requirement.⁴⁹ It allows the MiniMax M3 model to simultaneously hold the entire web-app-consolidation-plan.md, the operational definitions of the twelve NCCA subject agents, the intricacies of the Dagster 5-layer component

architecture, the complete Storybook Component Manifest, and the Design Token JSON registries in its active memory without suffering from context degradation or hallucination.¹ This immense capacity enables extended coding sessions lasting upwards of eight hours, where the model can break complex migrations—such as moving the hono-api into the new monorepo structure—down into subtasks, execute them concurrently, and continuously validate its own output without losing the thread of the overarching objective.⁴⁷

On autonomous software engineering and agentic benchmarks, the MiniMax M3 model achieves frontier-level performance. It scores an impressive 59.0% on the rigorous SWE-Bench Pro, 66.0% on Terminal-Bench 2.1, and 74.2% on MCP Atlas.⁵² These metrics indicate that M3 does not passively generate code snippets upon request; it acts as a reliable, proactive collaborative partner capable of autonomous terminal execution, long-term architectural planning, and dynamic tool invocation.⁴⁹

To harness this formidable power optimally during the web consolidation process, the API and system prompt parameters must be tuned precisely. MiniMax officially recommends setting the temperature parameter to 1.0, top_p to 0.95, and top_k to 40 to maximize creative problem-solving and flexible reasoning while maintaining logical coherence.⁵³ Furthermore, when defining system prompts for expansive, long-horizon operations like the UI/UX pipeline integration, explicit instructions must be provided to guide the model's pacing. The recommended system prompt explicitly dictates: "This is a very lengthy task. Make full use of the available context window. Complete each part thoroughly before continuing, and avoid exhausting tokens before the task is complete".⁵⁴

Terminal Orchestration via OpenCode and MCP Registry Configuration

To successfully bridge the massive cognitive capabilities of the MiniMax M3 model with the repository's localized execution environment, a robust terminal middleware is required. The OpenCode terminal agent serves as this necessary execution layer. OpenCode is an open-source, terminal-based AI coding agent specifically designed to support Model Context Protocol (MCP) servers natively, allowing developers to execute complex AI workflows directly from the command line without relying on external web interfaces.⁵⁵

Unlike other AI assistants that utilize standard mcp.json or mcpServers configuration formats, OpenCode requires MCP servers to be explicitly mapped under the mcp key within a specific opencode.json (or opencode.jsonc) file.⁵⁵ This configuration file can be located either in the user's global configuration directory or within the project root, with the project root configuration correctly taking precedence.⁵⁵

OpenCode Config Key	Tool Designation	Security Permission Logic	Functional Description
---------------------	------------------	---------------------------	------------------------

bash	Terminal Execution	"allow", "ask", or "deny"	Executes shell commands in the project environment (e.g., npm install, vitest). ⁵⁹
edit	File Modification	"allow", "ask", or "deny"	Modifies existing files using exact string replacements. ⁵⁹
write	File Creation	Controlled by edit permission	Creates new files or overwrites existing ones. ⁵⁹
read	File Ingestion	"allow", "ask", or "deny"	Reads file contents directly from the codebase into the LLM context. ⁵⁹

By defining local servers within the `opencode.json` utilizing `"type": "local"` and specifying precise command arrays, the repository can seamlessly inject both the Storybook MCP server and the necessary file system MCP servers directly into the MiniMax M3 context.⁵⁵ For example, integrating the official Storybook MCP would involve a command array utilizing `npx -y @storybook/addon-mcp`.⁵⁸ Furthermore, OpenCode features a highly granular permission system that allows architects to explicitly "allow", "ask", or "deny" specific actions like bash execution or file edit capabilities.⁵⁹ By configuring these parameters—alongside setting specific agent temperature overrides within the configuration schema—the engineering team ensures the agent remains strictly governed by repository rules while possessing the operational autonomy necessary to write, compile, and test the new UI/UX code architecture.⁵⁹

Master Implementation Prompt for MiniMax M3 Integration

To initiate this massive architectural integration, a highly specific, context-rich prompt must be fed to the MiniMax M3 model via the OpenCode terminal agent. This prompt must encapsulate the entire context of the cianfhoghclaim repository, the strict sequential rules of the web consolidation plan, the implementation logic of the Storybook MCP, the establishment of the Style Dictionary token pipeline, and the configuration of Dagger for continuous testing. The integration strategy relies on providing the agent with unambiguous directives that capitalize on its one-million token context window, ensuring it completes the setup of the machine-readable infrastructure before attempting the first code migration. The prompt

should be injected into the MiniMax M3 system prompt or passed as the initial user directive within the OpenCode interface:

System Directive:

You are a Staff-Level UI/UX Architect and AI Integration Specialist operating autonomously on the cianfhoghlaime monorepo. This is a very lengthy task. Make full use of your 1,000,000 token context window. Complete each part thoroughly before continuing, and avoid exhausting tokens before the task is complete.

Your objective is to establish a state-of-the-art Agentic Design System using Infrastructure as Code (IaC) principles, strictly preparing the frontend for the seamless, sequential execution of the Web App Consolidation Plan. You will leverage Storybook MCP, Style Dictionary v4, Dagger CI/CD, and your autonomous bash/edit capabilities to build this machine-readable infrastructure.

Execute the following phases strictly in order, validating success at each step using your bash and run-story-tests tools:

PHASE 1: AGENTIC INFRASTRUCTURE INITIALIZATION (OPENCODE & MCP)

1. In the root of the repository, create or update `opencode.json` to properly register the required Model Context Protocol (MCP) servers. The file must strictly adhere to the `https://opencode.ai/config.json` schema.
2. Ensure the `mcp` block includes the Storybook MCP server configured as a local stdio server using the command array `["npx", "-y", "@storybook/addon-mcp"]`.
3. Explicitly set permissions in the `opencode.json` to allow autonomous execution: `"permission": { "bash": "allow", "edit": "allow", "read": "allow" }`.
4. Set the default agent to a planning or building agent suitable for complex multi-step execution.

PHASE 2: DESIGN TOKENS AS CODE (STYLE DICTIONARY V4)

1. Initialize a central design token registry at `cianfhoghlaime/web/design-tokens/` containing a `tokens.json` file properly formatted to the Design Tokens Community Group (DTCG) specification (utilizing `$value` and `$type` nodes). Include core brand colors, typography, and spacing tailored for a Celtic educational interface.
2. Install Style Dictionary v4 and integrate the Tailwind CSS transformer.
3. Create an ESM-compliant Style Dictionary configuration (`sd.config.js`) that takes `tokens.json` as the source and outputs a Tailwind CSS v4 compliant configuration file using `formatType: 'js'`.
4. Execute the build process via your bash tool to verify that the tokens successfully compile into valid CSS variables and Tailwind utilities without errors.

PHASE 3: MACHINE-READABLE COMPONENT LIBRARY (STORYBOOK MCP)

1. Navigate to the target consolidated directory: `cianfhoghlaime/web/cio-web/`. Ensure the TanStack Start and React environment is properly initialized as per the consolidation plan.
2. Install Storybook for React via `npx storybook@latest init`.
3. Install the official MCP addon by executing `npx storybook add @storybook/addon-mcp`.
4. Register the MCP server locally using `npx mcp-add --type http --url "http://localhost:6006/mcp" --scope project`.
5. Generate a basic atomic component (e.g., `PrimaryButton.tsx`) using the newly created

Tailwind design tokens.

6. Write a Storybook story for this component and implement interaction tests using Vitest.
7. Autonomously trigger the Storybook MCP run-story-tests tool. If the test fails (e.g., due to accessibility contrast or missing props), utilize your self-healing loop: query the docs toolset, modify the component via the edit tool, and re-run the tests until they pass.

PHASE 4: DAGGER CI/CD INTEGRATION

1. Within the repository's existing Dagger pipeline definitions (e.g., `cianfhoghlaime/dagger/` or `.agents/skills/dagger/`), implement a new TypeScript module utilizing the Daggiverse SDKs.
2. Configure this Dagger module to spin up a sandboxed container that installs Node, mounts the `web/cio-web/` directory, and autonomously runs both ESLint and the Vitest/Storybook test suites.
3. Ensure this pipeline step is deterministic and caches dependencies optimally. Run `dg check toml` to ensure syntax validity in the surrounding architecture.

PHASE 5: DOMAIN 1 CONSOLIDATION PROOF-OF-CONCEPT

1. Consult the `web-app-consolidation-plan.md`. The first sequential target is the `hono-api` domain, representing the lowest operational risk.
2. Copy the contents of `cianfhoghlaime/web/hono-api/` into `cianfhoghlaime/web/cio-web/api/`.
3. Wire the Hono API routes into the TanStack Start server configuration within `web/cio-web/`.
4. Verify via terminal bash execution that the API gateway endpoints successfully resolve.
5. Provide a comprehensive summary of the state of the monorepo, detailing the successful implementation of the machine-readable infrastructure and the readiness for the Domain 2 (`croilar-web`) migration.

Future Outlook and Integration Implications

The transformation of the `cianfhoghlaime` repository from a backend-heavy data ecosystem into a fully consolidated, agentic UI/UX platform requires a profound operational shift. By transitioning away from isolated, human-readable design processes and aggressively embracing machine-readable infrastructure, the platform future-proofs its entire development lifecycle.¹² The sequential execution of the web application consolidation plan provides a structurally sound canvas for this shift, minimizing deployment risks while centralizing authentication systems and component libraries.⁵

While the migration of standard web interfaces like the `oideachais-web` and `croilar-portal` will benefit immediately from the TanStack and React-based Storybook MCP integration, the medium-risk `tuatha-ui` domain presents a unique architectural challenge.⁵ The Tuatha interface operates as an educational MMO built on Babylon.js and SpacetimeDB.² Integrating agentic design into a 3D web environment will require extending the token infrastructure beyond standard CSS mapping, ensuring that the 3D scene loader and MMO game states dynamically ingest the same underlying design variables defined in the Style Dictionary pipeline.⁵ Integrating the Storybook Model Context Protocol alongside the OpenCode terminal agent creates a critical, automated self-healing feedback loop.³² It allows AI agents to query

component manifests, generate complex UI code, run interaction tests, and correct their own syntax and accessibility errors autonomously before the code is ever committed.³² When coupled with a Style Dictionary v4 pipeline that treats design tokens as mathematically immutable code, visual consistency and strict adherence to the brand's guidelines are enforced instantaneously.¹⁹ Furthermore, migrating the CI/CD execution of these frontend validation tests into programmable Dagger containers guarantees deterministic, repeatable validation regardless of whether the code is executed locally by an agent or remotely on a cloud server.³⁵ At the center of this complex orchestration is the MiniMax M3 model. Its proprietary Sparse Attention architecture, optimized multimodal capabilities, and massive one-million token context window provide the sustained cognitive bandwidth necessary to internalize the entirety of the repository's complex state.⁴⁶ By bridging this powerful model with the local file system through the OpenCode terminal agent, and governing it with a meticulously crafted system prompt detailing strict operational phases, the cianfhoghlaím project can successfully operationalize state-of-the-art generative AI. This comprehensive integration strategy transforms the AI from a simple, reactionary coding assistant into a highly governable, autonomous, and profoundly reliable infrastructural partner capable of architecting the next generation of Celtic educational software.

Works cited

1. Activity · cianfhoghlaím/cianfhoghlaím - GitHub, accessed on July 12, 2026, <https://github.com/cianfhoghlaím/cianfhoghlaím/activity>
2. Cian Mac an Déisigh Uí Liatháin (Deacy-Lyons) cianfhoghlaím - GitHub, accessed on July 12, 2026, <https://github.com/cianfhoghlaím>
3. cianfhoghlaím/openspec/specs/dagster-5-layer-component ... - GitHub, accessed on July 12, 2026, <https://github.com/cianfhoghlaím/cianfhoghlaím/blob/main/openspec/specs/dagster-5-layer-component-architecture/spec.md>
4. cianfhoghlaím/openspec/AGENTS.md at main - GitHub, accessed on July 12, 2026, <https://github.com/cianfhoghlaím/cianfhoghlaím/blob/main/openspec/AGENTS.md>
5. cianfhoghlaím/docs/audit/web-app-consolidation-plan.md at main ..., accessed on July 12, 2026, <https://github.com/cianfhoghlaím/cianfhoghlaím/blob/main/docs/audit/web-app-consolidation-plan.md>
6. UI/UX: Our Selection of the Best Generative AI Tools of 2026 - aivancity, accessed on July 12, 2026, <https://aivancity.ai/en/blog/ui-ux-notre-selection-des-meilleurs-outils-ia-generatives-de-2026/>
7. Best 7 AI Tools for UI/UX Designers in 2026 | by Obiora O. | Medium, accessed on July 12, 2026, <https://medium.com/@zukkyzenith007/best-7-ai-tools-for-ui-ux-designers-in-2026-160a6782ce9b>

8. AI Agent Trends to Watch in 2026 - SotaTek ANZ, accessed on July 12, 2026, <https://sotatek.com.au/blogs/ai-agent-trends-2026/>
9. 15 Best AI Design Tools for UI/UX Designers in 2026 | by Ravidudilusha, accessed on July 12, 2026, <https://medium.muz.li/15-best-ai-design-tools-for-ui-ux-designers-in-2026-3531e51a893b>
10. Storybook MCP for React, accessed on July 12, 2026, <https://storybook.js.org/blog/storybook-mcp-for-react/>
11. Storybook MCP sneak peek, accessed on July 12, 2026, <https://storybook.js.org/blog/storybook-mcp-sneak-peek/>
12. Agentic Design Systems: The Complete Guide, accessed on July 12, 2026, <https://www.intodesignsystems.com/agentic-design-systems>
13. Agentic Design Systems in 2026 with Brad Frost - YouTube, accessed on July 12, 2026, <https://www.youtube.com/watch?v=Vq78K3t9KYc>
14. Front-End Developer Jobs 2026 | Vetted Contracts - Lemon.io, accessed on July 12, 2026, <https://lemon.io/for-developers/front-end-developer-jobs/>
15. The Agentic AI Infrastructure Landscape in 2025 — 2026: A Strategic Analysis for Tool-Builders | by Sri Srujan Mandava | Medium, accessed on July 12, 2026, <https://medium.com/@vinniesmandava/the-agentic-ai-infrastructure-landscape-i-n-2025-2026-a-strategic-analysis-for-tool-builders-b0da8368aee2>
16. Design Tokens Workflow for Visual Consistency Teams - Oleno, accessed on July 12, 2026, <https://oleno.ai/blog/design-tokens-workflow-maintain-visual-consistency-across-teams/>
17. Figma to iOS App: Convert Designs Into Production-Ready Apps, accessed on July 12, 2026, <https://www.rocket.new/blog/figma-to-ios-app-turn-designs-into-production-ready-apps>
18. Using Style Dictionary to transform Tailwind config into SCSS variables, CSS custom properties, and JavaScript via design tokens - DEV Community, accessed on July 12, 2026, https://dev.to/philw_/using-style-dictionary-to-transform-tailwind-config-into-scss-variables-css-custom-properties-and-javascript-via-design-tokens-24h5
19. Style Dictionary | Style Dictionary, accessed on July 12, 2026, <https://styledictionary.com/>
20. Style Dictionary V4 release plans - Tokens Studio, accessed on July 12, 2026, <https://tokens.studio/blog/style-dictionary-v4-plan>
21. Version 4 statement | Style Dictionary, accessed on July 12, 2026, <https://styledictionary.com/versions/v4/statement/>
22. cianfhoghlaime/agents/skills/tanstack-start/SKILL.md at main, accessed on July 12, 2026, <https://github.com/cianfhoghlaime/cianfhoghlaime/blob/main/agents/skills/tanstack-start/SKILL.md>
23. Design Tokens | Style Dictionary, accessed on July 12, 2026, <https://styledictionary.com/info/tokens/>

24. nado1001/style-dictionary-tailwindcss-transformer: This is a plugin to generate the config of Tailwind CSS using Style Dictionary - GitHub, accessed on July 12, 2026, <https://github.com/nado1001/style-dictionary-tailwindcss-transformer>
25. Structuring and Splitting Large-Scale Figma Design Systems: A 2025 Master Guide for Scalable, Maintainable, and Performant Component Libraries | by Claus Rainer Anton Nisslmüller | Medium, accessed on July 12, 2026, <https://medium.com/@claus.nisslmueller/structuring-and-splitting-large-scale-figma-design-systems-a-2025-master-guide-for-scalable-c1c3a7dabb0e>
26. Using Storybook with AI | Storybook docs, accessed on July 12, 2026, <https://storybook.js.org/docs/ai>
27. Storybook MCP Server: Enhancing Component Documentation - Practical AI, Copilot & Modern Development Insights, accessed on July 12, 2026, <https://aidevme.com/storybook-mcp-server-the-complete-developer-guide-to-ai-assisted-ui-component-development/>
28. Storybook MCP for AI-aware component libraries - LogRocket Blog, accessed on July 12, 2026, <https://blog.logrocket.com/storybook-mcp-component-libraries/>
29. How to test UIs with Storybook | Storybook docs, accessed on July 12, 2026, <https://storybook.js.org/docs/writing-tests>
30. Supercharge Your Design System with LLMs and Storybook MCP - Codrops, accessed on July 12, 2026, <https://tympanus.net/codrops/2025/12/09/supercharge-your-design-system-with-llms-and-storybook-mcp/>
31. Addon MCP | Storybook integrations, accessed on July 12, 2026, <https://storybook.js.org/addons/@storybook/addon-mcp>
32. MCP server | Storybook docs, accessed on July 12, 2026, <https://storybook.js.org/docs/ai/mcp/overview>
33. Storybook MCP Server, accessed on July 12, 2026, <https://mcpservers.org/servers/raksbisht/storybook-mcp>
34. mcpland/storybook-mcp: A MCP server for Storybook. - GitHub, accessed on July 12, 2026, <https://github.com/mcpland/storybook-mcp>
35. Dagger.io, accessed on July 12, 2026, <https://dagger.io/>
36. Dagger — CI/CD as Code that Runs Anywhere | by Vechtomova Maria | Marvelous MLOps, accessed on July 12, 2026, <https://medium.com/marvelous-mlops/dagger-ci-cd-as-code-that-runs-anywhere-fa25d3861b50>
37. Workflow runs · cianfhoghlaime/cianfhoghlaime - GitHub, accessed on July 12, 2026, <https://github.com/cianfhoghlaime/cianfhoghlaime/actions>
38. Daggervise, accessed on July 12, 2026, <https://daggervise.dev/>
39. dagger repositories - GitHub, accessed on July 12, 2026, <https://github.com/orgs/dagger/repositories>
40. Modules - Dagger Docs, accessed on July 12, 2026, <https://docs.dagger.io/0.21.7/modules>
41. Setup Guides - Dagger docs, accessed on July 12, 2026, <https://docs.dagger.io/next/modules/>
42. Automating UI testing and documentation with Storybook, Chromatic, and

- CircleCI, accessed on July 12, 2026,
<https://circleci.com/blog/automating-ui-testing-with-storybook-chromatic/>
43. cianfhoghlaime/agents/skills/dagster/references/cli/check.md at main · cianfhoghlaime/cianfhoghlaime · GitHub, accessed on July 12, 2026,
<https://github.com/cianfhoghlaime/cianfhoghlaime/blob/main/agents/skills/dagster/references/cli/check.md>
 44. cianfhoghlaime/agents/skills/dagster/references/components/state, accessed on July 12, 2026,
<https://github.com/cianfhoghlaime/cianfhoghlaime/blob/main/agents/skills/dagster/references/components/state-backed/using.md>
 45. MiniMax M3 IS INSANE! BEST Opensource AI Model! Beats Opus 4.7 and 50x Cheaper! (Fully Tested), accessed on July 12, 2026,
<https://www.youtube.com/watch?v=p6Npi-HBoRU>
 46. MiniMaxAI/MiniMax-M3 - Hugging Face, accessed on July 12, 2026,
<https://huggingface.co/MiniMaxAI/MiniMax-M3>
 47. Deploy Long-Context Reasoning and Agentic Workflows with MiniMax M3 on NVIDIA Accelerated Infrastructure | NVIDIA Technical Blog, accessed on July 12, 2026,
<https://developer.nvidia.com/blog/deploy-long-context-reasoning-and-agentic-workflows-with-minimax-m3-on-nvidia-accelerated-infrastructure/>
 48. minimaxai / minimax-m3 - NVIDIA API Documentation, accessed on July 12, 2026,
<https://docs.api.nvidia.com/nim/reference/minimaxai-minimax-m3>
 49. minimax-m3 - Ollama, accessed on July 12, 2026,
<https://ollama.com/library/minimax-m3>
 50. MiniMax M3 - Coding & Agentic Frontier, 1M Context, Multimodal, accessed on July 12, 2026, <https://www.minimax.io/models/text/m3>
 51. MiniMax M3 Released - Medium, accessed on July 12, 2026,
<https://medium.com/data-science-in-your-pocket/minimax-m3-released-377b225762dc>
 52. MiniMax M3: Frontier Coding, 1M Context, Native Multimodality — All in One Model, accessed on July 12, 2026, <https://www.minimax.io/blog/minimax-m3>
 53. MiniMax M3 - How to Run Locally | Unsloth Documentation, accessed on July 12, 2026, <https://unsloth.ai/docs/models/minimax-m3>
 54. M-series Usage Tips - MiniMax API Docs, accessed on July 12, 2026,
<https://platform.minimax.io/docs/token-plan/prompting-best-practices>
 55. OpenCode | Products - Documentation - IONOS, accessed on July 12, 2026,
<https://docs.ionos.com/cloud/ai/mcp-server/connect-to-an-ai-client/opencode>
 56. github-mcp-server/docs/installation-guides/install-opencode.md at main, accessed on July 12, 2026,
<https://github.com/github-mcp-server/blob/main/docs/installation-guides/install-opencode.md>
 57. [FEATURE]:common mcpServers json format support · Issue #28364 · anomalyco/opencode, accessed on July 12, 2026,
<https://github.com/anomalyco/opencode/issues/28364>
 58. How to Add MCP Servers to opencode, accessed on July 12, 2026,

<https://mcp.directory/clients/opencode>

59. Tools | OpenCode, accessed on July 12, 2026, <https://opencode.ai/docs/tools/>

60. Agents - OpenCode, accessed on July 12, 2026, <https://opencode.ai/docs/agents/>

61. Reimagining tech infrastructure for agentic AI - McKinsey, accessed on July 12, 2026,

<https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/reimagining-tech-infrastructure-for-and-with-agentic-ai>